

# autolisp-file-compatible Usage Guide

Codex

June 24, 2026

## Contents

|                                            |           |
|--------------------------------------------|-----------|
| <b>1 Purpose</b>                           | <b>1</b>  |
| <b>2 What The Module Does</b>              | <b>1</b>  |
| <b>3 Scenario Files</b>                    | <b>2</b>  |
| <b>4 Scenario Kinds</b>                    | <b>3</b>  |
| 4.1 Roundtrip Scenarios . . . . .          | 3         |
| 4.2 Builtin Scenarios . . . . .            | 4         |
| <b>5 Scenario Metadata</b>                 | <b>6</b>  |
| <b>6 Value Forms For Builtin Scenarios</b> | <b>6</b>  |
| <b>7 Command-Line Tool</b>                 | <b>7</b>  |
| <b>8 Typical Invocations</b>               | <b>9</b>  |
| <b>9 Report Shape</b>                      | <b>10</b> |
| <b>10 Current Limits</b>                   | <b>11</b> |

## 1 Purpose

This document explains how to use the current `autolisp-file-compatible` implementation.

It is practical user documentation for the scenario harness and command-line driver. Design rationale remains in:

- `documentation/design.org`

## 2 What The Module Does

`autolisp-file-compat` is a compatibility-audit harness for:

- file I/O behavior,
- pathname and search-path behavior,
- printer and read-back behavior.

It is intended to support:

- local regression testing,
- SBCL versus CCL portability checks,
- later comparison against real AutoLISP products.

## 3 Scenario Files

The harness is driven by declarative `=.sexp=` scenario files under:

```
#+TEXINFO_FILENAME: usage.info
#+TEXINFO_DIR_CATEGORY: clautolisp
#+TEXINFO_DIR_TITLE: autolisp-file-compat: (usage).
#+TEXINFO_DIR_DESC: AutoLISP file-compat programmer's guide.
clautolisp/autolisp-file-compat/scenarios/
```

The current corpus includes families such as:

- `newlines/`
- `encodings/`
- `paths/`
- `mutations/`
- `printers/`
- `streams/`

The current stream/printer corpus specifically includes:

- write/read-line roundtrips,
- append and successive EOF checks,
- write-char/read-char sequences,
- explicit OPEN encoding arguments such as "utf-8",
- unsupported OPEN encoding arguments,
- negative open-for-read cases on missing files,
- file-targeted printer output and printer-to-READ roundtrips,
- line-by-line read-back validation of printer-produced files,
- append workflows with explicit encodings such as UTF-8,
- relative OPEN and append workflows resolved through :current-directory,
- file-targeted PRINC roundtrips through READ under :current-directory,
- backslash-delimited relative subpaths resolved through :current-directory,
- file-targeted PRIN1 compound-form roundtrips under :current-directory,
- current-directory-relative VL-FILE-SIZE and VL-FILE-DELETE cases,
- current-directory-relative VL-FILE-RENAME cases, including backslash subpaths,
- negative current-directory VL-FILE-RENAME cases,
- current-directory-relative VL-FILE-COPY cases,
- negative current-directory VL-FILE-COPY cases,
- current-directory-relative VL-FILENAME-MKTEMP cases,
- the leading blank line introduced by file-targeted PRINT.

## 4 Scenario Kinds

There are currently two scenario kinds.

## 4.1 Roundtrip Scenarios

These write text or bytes, read them back, and compare:

- bytes,
- decoded text,
- decoded lines.

Typical fields:

- `:kind :roundtrip`
- `:relative-path`
- `:input-text` or `:input-bytes`
- `:expected-text`
- `:expected-lines`
- `:expected-bytes`

Example:

```
(:name "lf-text"
 :kind :roundtrip
 :classification :portable
 :tags (:text :newline :lf)
 :relative-path "artifacts/lf-text.txt"
 :external-format :utf-8
 :newline-mode :lf
 :input-text "one
two
"
 :expected-text "one
two
"
 :expected-lines ("one" "two"))
```

## 4.2 Builtin Scenarios

These create a temporary workspace, install setup fixtures, configure path state, call an AutoLISP builtin through the runtime function-object layer, and compare the resulting runtime value.

Typical fields:

- `:kind :builtin`
- `:setup-files`
- `:current-directory`
- `:support-paths`
- `:trusted-paths`
- `:builtin-name`
- `:arguments`
- `:expected-value`
- optional `:artifact-relative-path` and artifact expectations

Builtin scenarios may also use a multi-step form instead of a single builtin call.

Additional fields for that form are:

- `:steps`
- optional per-step `:bind`
- optional per-step `:expected-value`
- optional scenario-level `:result-ref`

Example:

```
(:name "findfile-basic"
:kind :builtin
:classification :portable
:tags (:builtin :paths :findfile)
:setup-files ((:type :directory :relative-path "support/")
               (:relative-path "support/example.txt" :input-text "hello")))
```

```

:current-directory "cwd/"
:support-paths ("support/")
:builtin-name "FINDFILE"
:arguments ("example.txt")
:expected-value (:workspace-relative "support/example.txt"))

```

Multi-step example:

```

(:name "open-write-read-line-basic"
:kind :builtin
:classification :portable
:tags (:builtin :stream :file-descriptor)
:steps ((:builtin-name "OPEN"
          :arguments ("sample.txt" "w")
          :bind "out")
         (:builtin-name "WRITE-LINE"
          :arguments ("alpha" (:ref "out"))))
         (:builtin-name "CLOSE"
          :arguments ((:ref "out"))))
         (:builtin-name "OPEN"
          :arguments ("sample.txt" "r")
          :bind "in")
         (:builtin-name "READ-LINE"
          :arguments ((:ref "in"))
          :bind "line"))
:result-ref "line"
:expected-value (:string "alpha")
:artifact-relative-path "sample.txt"
:expected-text "alpha
")

```

## 5 Scenario Metadata

Each scenario may also carry:

- :name
- :description
- :classification

- `:tags`

Current classifications are:

- `:portable`
- `:implementation-sensitive`
- `:host-sensitive`
- `:product-sensitive`
- `:unknown`

Tags are free-form keywords used for filtering.

## 6 Value Forms For Builtin Scenarios

Builtin scenario arguments and expected values may use plain Lisp literals, or the harness-specific value forms below.

Supported forms include:

- `(:string "abc")`
- `(:integer 17)`
- `(:real 3.5)`
- `(:symbol "FOO")`
- `(:nil)`
- `(:list 1 (:symbol "FOO") "bar")`
- `(:cons 1 2)`
- `(:workspace-relative "support/example.txt")`
- `(:ref "bound-step-name")` inside multi-step builtin arguments

These are mapped to the current AutoLISP runtime model before comparison.

## 7 Command-Line Tool

The user-facing command-line wrapper is:

`clautolisp/autolisp-file-compat/tools/run-file-compat/run-file-compat`

The `clautolisp` subproject Makefile also provides convenience targets:

- `make -C clautolisp run-file-compat`
- `make -C clautolisp run-file-compat-sbcl`
- `make -C clautolisp run-file-compat-ccl`
- `make -C clautolisp run-file-compat-autocad`
- `make -C clautolisp run-file-compat-bricscad`
- `make -C clautolisp audit-file-compat-autocad`
- `make -C clautolisp audit-file-compat-bricscad`

Useful variables:

- `FILE_COMPAT_SCENARIOS`
- `FILE_COMPAT_ARGS`

Product-backed runs use runner-template environment variables:

- `AUTOCAD_RUNNER`
- `BRICSCAD_RUNNER`

Each template is a shell command string that may reference:

- `__PROBE_FILE__`
- `__WRAPPER_FILE__`
- `__RESULT_FILE__`
- `__RESULT_DIR__`

Supported wrapper modes:



- `--sbcl`
- `--ccl`
- `--autocad`
- `--bricscad`

Supported driver options:

- `--runner-label MODE`
- `--classification CLASS`
- `--tag TAG`
- `--report-format FORMAT`
- `--output PATH`
- `--help`

MODE is one of:

- `local`
- `sbcl`
- `ccl`
- `autocad`
- `bricscad`

FORMAT is one of:

- `sexp`
- `json`

## 8 Typical Invocations

Run a single scenario file:

```
clautolisp/autolisp-file-compat/tools/run-file-compat/run-file-compat \
--sbcl \
--report-format json \
clautolisp/autolisp-file-compat/scenarios/basic-text-roundtrip.sexp
```

Run a whole scenario directory recursively:

```
clautolisp/autolisp-file-compat/tools/run-file-compat/run-file-compat \
--sbcl \
--report-format json \
clautolisp/autolisp-file-compat/scenarios
```

Run only builtin scenarios:

```
clautolisp/autolisp-file-compat/tools/run-file-compat/run-file-compat \
--sbcl \
--tag builtin \
--report-format json \
clautolisp/autolisp-file-compat/scenarios
```

Equivalent Makefile invocation:

```
make -C clautolisp run-file-compat \
FILE_COMPAT_ARGS="--tag builtin --report-format json"
```

Write JSON output into the repository artifact directory:

```
make -C clautolisp run-file-compat-sbcl \
FILE_COMPAT_ARGS="--tag builtin --report-format json --output .artifacts/file-compat"
```

Run only portable path scenarios:

```
clautolisp/autolisp-file-compat/tools/run-file-compat/run-file-compat \
--sbcl \
--classification portable \
--tag paths \
--report-format json \
clautolisp/autolisp-file-compat/scenarios
```

Write the combined report to a file:

```
clautolisp/autolisp-file-compat/tools/run-file-compat/run-file-compat \
--ccl \
--tag builtin \
--report-format json \
--output /tmp/file-compat-report.json \
clautolisp/autolisp-file-compat/scenarios
```

Run a product-backed audit through BricsCAD:

```
BRICSCAD_RUNNER="/Applications/BricsCAD V26.app/Contents/MacOS/bricscad" "__PROBE_FI
make -C clautolisp audit-file-compat-bricscad \
FILE_COMPAT_ARGS="--output /tmp/file-compat-bricscad.json"
```

## 9 Report Shape

The emitted result is a top-level report object with two fields:

- `:summary`
- `:reports`

The summary contains aggregate counts for:

- total scenarios,
- passed scenarios,
- failed scenarios,
- total checks,
- passed checks,
- failed checks.

Each individual report contains:

- the runner label,
- the scenario metadata,
- the check list,
- an optional artifact record.

## 10 Current Limits

The current implementation does not yet provide:

- a generic multi-step action language,
- full evaluator-driven scenario execution,
- full product support for every host-state-sensitive scenario.

The current builtin corpus is intentionally narrow and focused on deterministic file/path/printer checks that already exist in the implementation.

At present, real-product audits still require an external runner command for AutoCAD or BricsCAD. The adapter layer and its regression tests exist, but the actual product audit remains dependent on a configured local product installation or runner bridge.